UNIVERSITY COLLEGE OF ENGINEERING, PANRUTI
(A Constituent College of Anna University, Chennai)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AN INTELLIGENT AI-BASED CAMPUS PLACEMENT AND RESUME ANALYSIS SYSTEM

PRESENTED BY
AGADESHWARAN R (422622104003)
JOEL M (422622104015)
BALAJI P (422622104030)
DIVYA A (422622104703)

GUIDED BY
Dr. A. RAMACHANDRAN M.E,M.B.A,Ph.D
ASSISTANT PROFESSOR
DEPARTMENT OF CSE

ABSTRACT

Campus placements are very important for students to start their careers. In traditional placement systems, resumes are checked manually, which takes a lot of time and is not very efficient. Our project introduces an AI-Based campus placement and resume Analysis System that makes the recruitment process faster, fairer, and more transparent than manual resume screening.

The system includes a Professional Ranking System (PRS) that gives scores to candidates in an objective way using NLP and TF-IDF. It also uses Explainable AI (XAI), which clearly explains why the AI makes each decision. In addition, the platform has a gamified Cognitive Assessment Hub to test students' soft skills in five areas: Logic, Memory, Reaction Time, Spatial Awareness, and Problem Solving.

INTRODUCTION

1.Campus recruitment receives **thousands of applications for limited jobs**, making selection difficult.

2.Traditional systems rely on **manual resume screening**, which is time-consuming, subjective, and inefficient.

3.Students face **unclear skill gaps** and **lack transparency** in selection decisions.

4.This project proposes an **AI-Based placement platform** using NLP, ranking, XAI, and cognitive assessments.

5.The system **extracts skills using spaCy and TF-IDF** and ranks candidates with a multi-factor model.

6.It evaluates **soft skills through gamified assessments** and explains decisions using **XAI**.

7.A **scalable architecture** supports real-time analysis and multiple users, ensuring fair and reliable results.

“An Intelligent AI-based Campus Placement and Resume Analysis System”

1.Intelligent

1. The system does not just store resumes; it understands and evaluates them.
2. It smartly ranks students using multiple factors like skills and projects.

2.AI-based

1. Uses Artificial Intelligence and NLP to read resumes automatically.
2. Extracts skills and experience without manual work and matches them to jobs.

3.Campus Placement

1. Designed specifically for college placements.
2. Used by students, recruiters, and placement officers for hiring and tracking.

4.Resume Analysis

1. The system analyzes resumes to find skills, education, and certifications.
2. It also tells students which skills they are missing.

5.System

1. It is a complete end-to-end solution, not a single tool.
2. Includes frontend, backend, AI engine, and database working together.

DETAILED LITERATURE SURVEY

Feature	Paper 1: AI-Driven Job Rec (Apurba Nandi Avik Kumar Das)	Paper 2: AI-Powered Resume Feedback (Abhishek Mishra Siddharth Singh)	Paper 3: Automated Resume Screening (CH Sai Surya GLVS Kiriti)	Paper 4: Resume Analysis & Job Rec (Amruta Mankawade Vithika Pungliya)	Paper 5: Skill Rec & Resume Analysis (Riddhi R. Mirajkar Gitanjali R. Shinde)	Our Project: AN INTELLIGENT AI-BASED CAMPUS PLACEMENT AND RESUME ANALYSIS SYSTEM
Core NLP Model	BERT-based NER + Sentence-BERT	Fine-tuned BERT model	TF-IDF with KNN/SVM classification	TF-IDF + Naive Bayes classifier	Rule-based NLP extraction	spaCy NER + TF-IDF + Multi-factor Professional Ranking System
Skill Extraction Accuracy	88–92% (NER-based)	Approximately 81.6% (BERT accuracy)	98–99% (SVM classification)	97% (Naive Bayes)	Not explicitly quantified	92% accuracy with dynamic skill database
Matching Algorithm	Cosine Similarity + Multi-Armed Bandit	BERT embedding similarity	TF-IDF + KNN distance metrics	Cosine Similarity matching	Keyword-based matching	TF-IDF + Weighted PRS + Industry Trend Multipliers
Explainability (XAI)	Not implemented	Not implemented	Not implemented	Not implemented	Not implemented	Implemented with Rule-based NLG + Human-readable justifications
Soft Skills Assessment	Not implemented	Not implemented	Not implemented	Not implemented	Basic implementation via skill recommendations	Comprehensive implementation with Gamified Cognitive Hub across 5 dimensions
Professional Ranking	MAB-based ranking optimization	BERT similarity scoring	Category classification system	Profession prediction model	Field recommendation engine	Professional Ranking System (0–100%) with 6 weighted factors + Tier classification
Multi-Language Support	Implemented via MarianMT translation	Not implemented	Not implemented	Not implemented	Not implemented	English-only currently, but designed for scalability

DETAILED LITERATURE SURVEY

Feature	Paper 1: AI-Driven Job Rec	Paper 2: AI-Powered Resume Feedback	Paper 3: Automated Resume Screening	Paper 4: Resume Analysis & Job Rec	Paper 5: Skill Rec & Resume Analysis	Our Project: AN INTELLIGENT AI-BASED CAMPUS PLACEMENT AND RESUME ANALYSIS SYSTEM
Real-Time Analytics	Implemented with heatmaps and ROC-AUC	Not implemented	Basic dashboard implementation	Implemented with web scraping + job listing	Implemented with charts and field recommendations	Interactive dashboards with radar charts and trend analysis
Industry Trend Integration	Implemented via skill frequency analysis	Not implemented	Not implemented	Not implemented	Not implemented	Implemented with AI/ML ×1.25, Cloud ×1.15 multipliers
Feedback to Candidate	Not implemented	Implemented with skill gap identification	Not implemented	Implemented with job recommendations	Implemented with skill and course recommendations	Comprehensive feedback with XAI + Skill gaps + Upskilling paths
Scalability	High (MAB, SBERT architecture)	Medium (BERT model constraints)	Medium (SVM limitations)	Low (Naive Bayes constraints)	Low (Rule-based limitations)	High (Microservices architecture with MongoDB Atlas and Fast API)
Deployment Architecture	Monolithic (implied)	Not specified	Web application (implied)	Flask-based framework	Streamlit web application	Full-stack implementation (React + Node.js + FastAPI + MongoDB)
Cognitive Assessment	Not implemented	Not implemented	Not implemented	Not implemented	Not implemented	Implemented with 5 gamified cognitive modules
Key Innovation	Real-time skill identification with MAB optimization	Candidate feedback mechanism using BERT	High-accuracy traditional ML classification	Job market integration via web scraping	Skill recommendation system	Integrated ecosystem with XAI, cognitive assessment, and trend-aware ranking

PROBLEM STATEMENT

1.Recruitment Bias

- Human screening depends on personal judgment.
- Resumes may be rejected due to format or appearance.

2.High Application Volume

- Companies receive 1000+ resumes per role.
- Manual screening is slow and impractical.

3.Ineffective ATS Systems

- Existing ATS use basic keyword matching.
- Skilled candidates are often missed.

4.No Feedback to Students

- Students don’t know why they were rejected.
- Skill gaps remain unclear.

PROPOSED SOLUTION

1.AI-Based Objective Evaluation

- Resumes evaluated based on skills and experience.
- Eliminates human bias.

2.Professional Ranking System (PRS)

- Uses multi-factor scoring.
- Considers skills, projects, experience, and semantics.

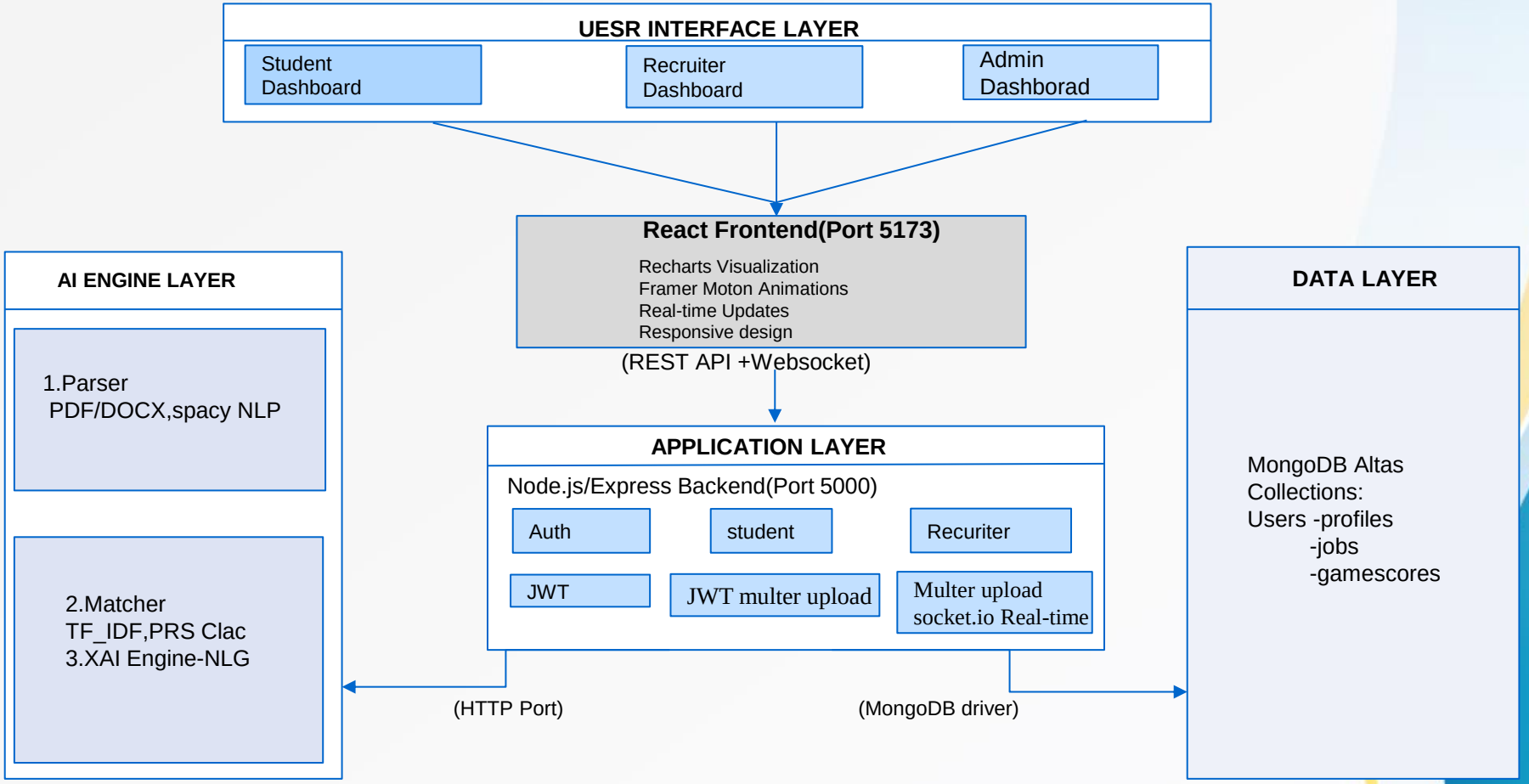
3.Explainable AI (XAI)

- Provides clear reasons for scores and rejections.
- Improves transparency and trust.

4.Scalable Modern Platform

- Handles large volumes efficiently.
- Built with modern, real-time technologies.

METHODOGY AND SYSTEM ARCHITECTURE



Four-Layer Architecture of AI-Powered Student Placement Portal:

1.User Interface Layer: React frontend with dashboards and interactive visuals.

2.Application Layer: Node.js/Express backend for authentication, uploads, and real-time communication.

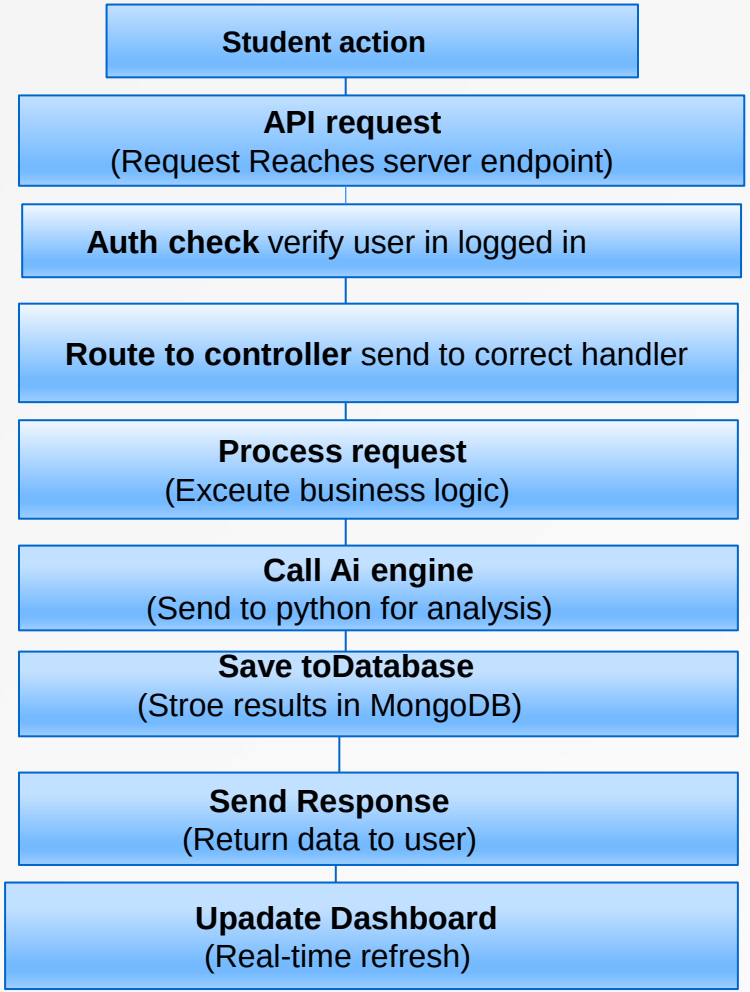
3.AI Engine Layer: Python–FastAPI engine using NLP for resume parsing, PRS ranking, and explainable results.

4.Data Layer: MongoDB Atlas for secure cloud data storage.

ALGORITHMS USED:

- **Algorithm 1:** Natural Language Processing (NLP) with spaCy.
- **Algorithm 2:** Weighted Multi-Factor Scoring .
- **Algorithm 3:** TF-IDF & Cosine Similarity.

Application Layer = Traffic Controller + Logic Processor + Data Manager



AI Engine (The Brain)

Technology: Python with FastAPI

This is the main part of the system that reads resumes, compares them with jobs, and gives scores.

1.main.py (Runs the AI system)

FastAPI: A fast Python framework used to build the AI service.

Uvicorn: Runs the FastAPI application so it can handle requests.

Example: This file makes the AI engine work and respond to requests.

2.parser.py (Reads the Resume)

pdfplumber: Extracts text clearly from PDF resumes.

spaCy (NLP library):

- Understands words, names, and sentence structure.
- Finds important details like skills, education, and experience.
- Uses PhraseMatcher to quickly identify technical skills.

Example: This file reads and understands the resume like a human.

3.matcher.py (Compares and Scores)

TF-IDF: Converts resume and job description into numbers and focuses on important words.

Cosine Similarity: Checks how closely the resume matches the job description.

Example: This file checks how well a student's resume matches a job and gives a similarity score.

Algorithm 1: Resume Parsing (The Reader)

- **Name**: Natural Language Processing (NLP) with spaCy.
- Why use this?:

  1.A PDF is just a picture of text to a computer. It doesn't know where the "Skills" section is.
  2.We need a smart way to find words like "Python" even if they are hidden in a paragraph.
  3.Simple Keyword Matching fails: If a student writes "Artificial Intelligence", a simple match for "AI" might miss it. NLP understands they are the same thing.

- How it works:

  1.It converts the PDF to plain text.
  2.The spaCy brain reads the text word-by-word.
  3.It looks for **Entities** (specific technical terms) using a **PhraseMatcher**.
  4.It extracts a clean list: ['Python', 'React', 'MongoDB'].

Algorithm 2: Professional Ranking System (The Judge)

- **Name**: Weighted Multi-Factor Scoring.
- Why use this?:

1.Most systems just say "Pass/Fail". That is too simple.
2.We want to know *how good* a student is.
3.Weighting is key: "Core Skills" (like coding) should matter more than "Hobbies".

- How it works:

It calculates 5 sub-scores:

1.Core Skills (35%): Do you have the exact skills the job needs?
2.Secondary Skills (15%): Do you have bonus skills?
3.Project Relevance (15%): Have you built projects similar to the job?
4.Semantic Match (10%): Does your resume "sound" like a good fit?
5.Experience (15%): Do you have work history?

It sums these up to get a final score out of 100.
Result: A student with 85/100 is a "Expert", while 40/100 is a "Learner".

Algorithm 3: Semantic Matching (The Context)

- **Name**: TF-IDF & Cosine Similarity.
- Why use this?:
    - Keywords are not enough.
    - If a job needs "Web Development" and a student writes "I built a website", a keyword search might fail.
    - Semantic Matching understands context.
- How it works:

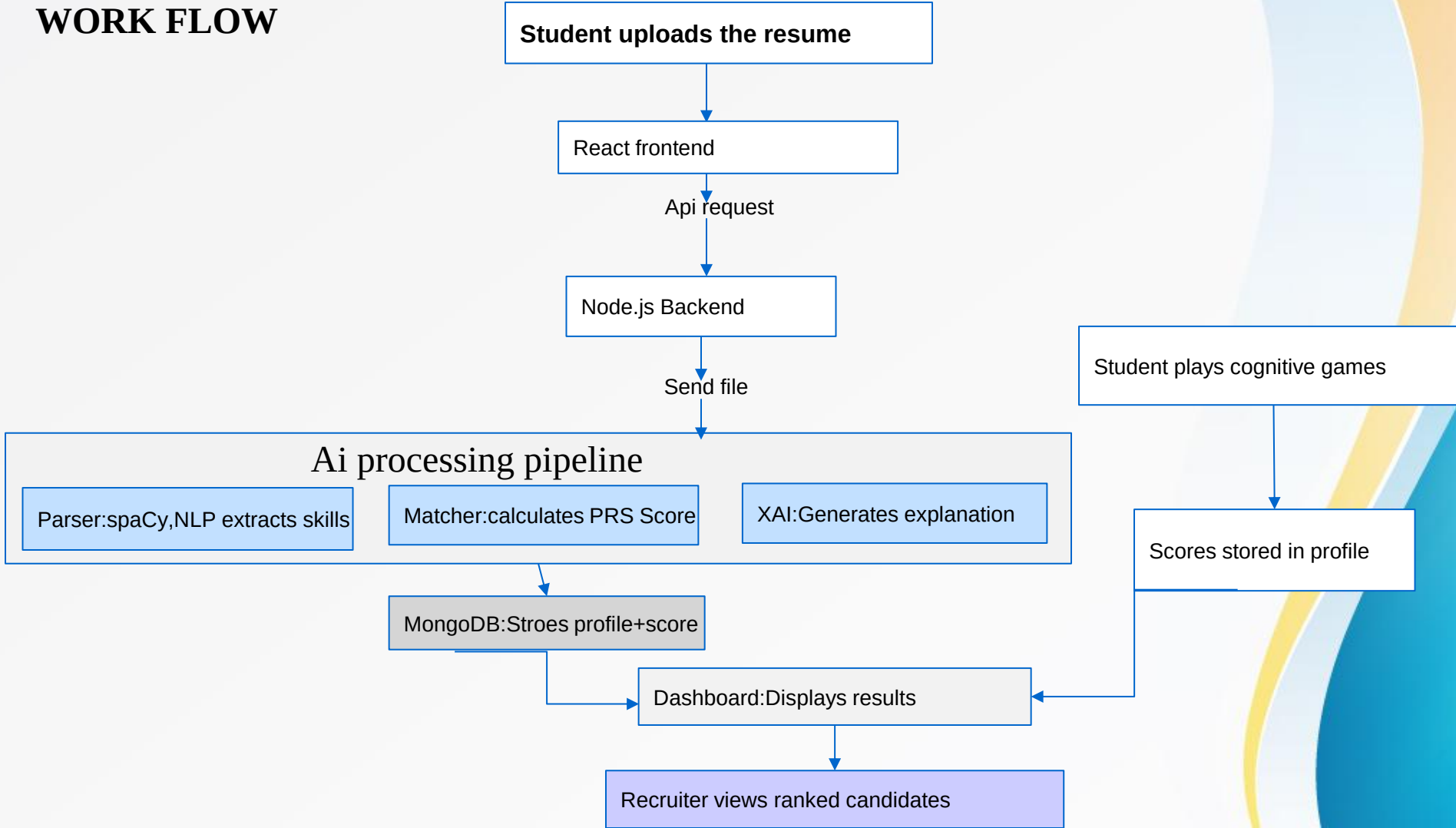
    TF-IDF: Determines which words are important (rare) and which are junk (common words like "the").

    1.Vectorization: Turns the Resume and Job Description into mathematical arrows (Vectors).

    2.Cosine Similarity: Measures the angle between the two arrows.

    3.Result: If the arrows point the same way, the match is high, even if the words are slightly different.

WORK FLOW


Student uploads the resume
React frontend
Api request
Node.js Backend
Send file
Ai processing pipeline
Parser:spaCy,NLP extracts skills
Matcher:calculates PRS Score
XAI:Generates explanation
MongoDB:Stroes profile+score
Student plays cognitive games
Scores stored in profile
Dashboard:Displays results
Recruiter views ranked candidates

HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS

- Processor: Intel i3 or higher
- RAM: 8 GB
- Hard Disk: 256 GB
- Internet Connection

SOFTWARE REQUIREMENTS

- Frontend: React, React-dom, React-router-dom, axios, recharts, framer-motion,lucide-react, vite.
- Backend: Express, Mongoose, Jsonwebtoken, bcryptjs,Multer, Socket.io, axios, dotenv, nodemailer.
- Database: MongoDB
- AI: fastapi, uvicorn, spacy, scikit-learn, pdfplumber, python-docx, numpy
- OS: Windows / Linux
- Tools: VS Code, Browser, npm, Python, Node.js ,Postman

WORK PLAN

Phase	Duration	Start Date	End Date	Deliverables
Phase 1: Requirements & Design	1 week	Day 1	Day 7	Requirements doc, System design, Database schema
Phase 2: Backend Development	1 week	Day 1	Day 7	REST APIs, Authentication, Database integration
Phase 3: AI Engine Development	1 week	Day 1	Day 7	Resume parser, PRS algorithm, XAI module
Phase 4: Frontend Development	1 week	Day 1	Day 7	Dashboard, Gaming Hub, Analytics
Phase 5: Integration & Testing	1 week	Day 1	Day 7	End-to-end testing, Bug fixes
Phase 6: Deployment & Documentation	2 weeks	Week 1	Week 2	Cloud deployment, User manual, API docs
Total Duration	**7 weeks**	-	-	-


Thank
you!